

Document number: P0958R2
Date: 2020-08-21
Project: Programming Language C++
Audience: SG4 - Networking
Reply-to: Christopher Kohlhoff <chris@kohlhoff.com>

Networking TS changes to support proposed Executors TS

Introduction

The purpose of this paper is to illustrate the changes to the Networking TS to conform to the proposed Executors TS in P0443R13.

All changes are relative to N4711.

Summary of changes

This paper proposes the following changes to the Networking TS:

- [Add a reference to the executors proposal.](#)
- [Remove `executor\_work\_guard` and make `work\_guard`, as these have been superseded by the executors proposal's `execution::outstanding\_work` property.](#)
- [Remove the `Executor` type requirements and the `is\_executor` type trait.](#)
- [Specify the "Requirements on asynchronous operations" in terms of the new executors model.](#)
- [Update `system\_executor` and `system\_context` to conform to the new executors model.](#)
- [Remove the polymorphic wrapper `executor` and replace it with a type alias for the executors proposal's `execution::any\_executor`.](#)
- [Specify the free functions `dispatch`, `post`, and `defer` in terms of the new executors model.](#)
- [Update the strand adapter to conform to the new executors model.](#)
- [Update the `use\_future\_completion` token to conform to the new executors model.](#)
- [Update `io\_context` to conform to the new executors model.](#)

In addition, this paper proposes [a minor extension to the executors proposal in P0443R13](#). This extension enables the use of the polymorphic wrapper `execution::any_executor` in the Networking TS.

Deployment experience

An implementation of the changes below, including a complete implementation of the executors-related specifications P0443R13, P1348R0, and P1393R0, can be found the Asio library at <https://github.com/chriskohlhoff/asio> and in Boost 1.74.

This implementation has been used to recompile libraries that depend on Asio, and has been tested in a number of applications. Some library modifications were required, but the majority of the applications needed minor or no modifications.

Proposed wording

Add a reference to the executors proposal

*Add a reference to the executors proposal in -5- **Namespaces and headers** [namespaces]:*

-2- Unless otherwise specified, references to other entities described in this Technical Specification are assumed to be qualified with `std::experimental::net::v1::`, references to entities described in the C++ standard are assumed to be qualified with `std::`, ~~and~~ references to entities described in C++ Extensions for Library Fundamentals are assumed to be qualified with `std::experimental::fundamentals_v2::`, and references to entities described in P1393R0 A General Property Customization Mechanism and in P0443R13 A Unified Executors Proposal for C++ are assumed to be qualified with `std::`.

Remove superseded `executor_work_guard` and `make_work_guard`

Remove `executor_work_guard` from -12.1- Header `<experimental/netfwd>` synopsis `[fwd.decl.synop]`:

```
template<class Executor>
class executor_work_guard;
```

Remove `executor_work_guard` and `make_work_guard` from -13.1- Header `<experimental/executor>` synopsis `[async.synop]`:

```
template<class Executor>
class executor_work_guard;

// 13.17, make_work_guard:

template<class Executor>
  executor_work_guard<Executor>
  make_work_guard(const Executor& ex);
template<class ExecutionContext>
  executor_work_guard<typename ExecutionContext::executor_type>
  make_work_guard(ExecutionContext& ctx);
template<class T>
  executor_work_guard<associated_executor_t<T>>
  make_work_guard(const T& t);
template<class T, class U>
  auto make_work_guard(const T& t, U& u)
  -> decltype(make_work_guard(get_associated_executor(t, forward<U>(u))));
```

Remove sections -13.16- Class template `executor_work_guard` `[async.exec.work.guard]` and -13.17- Function `make_work_guard` `[async.make.work.guard]` in their entirety.

Remove Executor requirements and `is_executor` type trait

Modify Table 3 - Template parameters and type requirements `[summary]` as follows:

Executor	executor (13.2.2) <u>execution::executor concept (P0443R13)</u>
----------	---

Remove `is_executor` and `is_executor_v` from -13.1- Header `<experimental/executor>` synopsis `[async.synop]`:

```
template<class T> struct is_executor;

template<class T>
constexpr bool is_executor_v = is_executor<T>::value;
```

Remove section -13.2.2- Executor requirements `[async.reqmts.executor]` in its entirety.

Modify Table 5 - `ExecutionContext` requirements `[async.reqmts.executioncontext]` as follows:

expression	return type	assertion/note pre/post-condition
X::executor_type	type meeting Executor-(13.2.2) requirements A type satisfying the <u>execution::executor concept (P0443R13)</u> .	

Modify section -13.2.7.8- I/O executor [*async.reqmts.async.io.exec*] as follows.

-1- An asynchronous operation has an associated executor satisfying the ~~Executor-(13.2.2) requirements~~execution::executor concept (P0443R13). If not otherwise specified by the asynchronous operation, this associated executor is an object of type `system_executor`.

[...]

-3- Let `Executor1` be the type of the associated executor. Let `ex1` be a value of type `Executor1`, representing the associated executor object obtained as described above. can_query_v<Executor1, execution::context t> shall be true, and std::query(ex1, execution::context t) shall yield a value of type execution_context& or of type E&, where E satisfies the ExecutionContext (13.2.3) requirements.

Modify section -13.2.7.9- Completion handler executor [*async.reqmts.async.handler.exec*] as follows.

-1- A completion handler object of type `CompletionHandler` has an associated executor satisfying the ~~Executor requirements (13.2.2)~~execution::executor concept (P0443R13). The type of this associated executor is `associated_executor_t<CompletionHandler, Executor1>`. Let `Executor2` be the type `associated_executor_t<CompletionHandler, Executor1>`. Let `ex2` be a value of type `Executor2` obtained by performing `get_associated_executor(completion_handler, ex1)`. can_query_v<Executor2, execution::context t> shall be true, and std::query(ex2, execution::context t) shall yield a value of type execution_context& or of type E&, where E satisfies the ExecutionContext (13.2.3) requirements.

Modify section -13.2.7.14- Composed asynchronous operations [*async.reqmts.async.composed*] as follows.

An intermediate operation's completion handler shall have an associated executor that is either:

- the type `Executor2` and object `ex2` obtained from the completion handler type `CompletionHandler` and object `completion_handler`; or
- an object of an unspecified type satisfying the ~~Executor requirements (13.2.2)~~execution::executor concept (P0443R13), that delegates executor operations to the type `Executor2` and object `ex2`.

Remove section -13.9- Class template *is_executor* [*async.is.exec*] in its entirety.

Modify section -13.10- Executor argument tag [*async.executor.arg*] as follows.

The `executor_arg_t` struct is an empty structure type used as a unique type to disambiguate constructor and function overloading. Specifically, types may have constructors with `executor_arg_t` as the first argument, immediately followed by an argument of a type that satisfies the ~~Executor requirements (13.2.2)~~execution::executor concept (P0443R13).

Modify section -13.12- Class template *associated_executor* [*async.assoc.exec*] as follows.

-1- Class template `associated_executor` is an associator (13.2.6) for ~~the Executor-(13.2.2) type requirements~~executors, with default candidate type `system_executor` and default candidate object `system_executor()`.

Modify Table 9 - *associated_executor specialization requirements* as follows:

Expression	Return type	Note
------------	-------------	------

typename X::type	A type meeting Executor requirements (13.2.2) A type satisfying the <u>execution::executor concept (P0443R13)</u> .	
------------------	---	--

Modify section -13.13- **Function `get_associated_executor` [`async.assoc.exec.get`]** as follows.

-3- **Remarks:** This function shall not participate in overload resolution unless ~~`is_executor_v<Executor> is true`~~`is_convertible<Executor&, execution context&>::value is false`.

Modify section -13.14- **Class template `executor_binder` [`async.exec.binder`]** as follows.

-1- The class template `executor_binder` binds executors to objects. A specialization `executor_binder<T, Executor>` binds an executor of type `Executor` satisfying the ~~Executor requirements (13.2.2)~~`execution::executor concept (P0443R13)` to an object or function of type `T`.

Modify section -13.15- **Function `bind_executor` [`async.bind.executor`]** as follows.

-2- **Remarks:** This function shall not participate in overload resolution unless ~~`is_executor_v<Executor> is true`~~`is_convertible<Executor&, execution context&>::value is false`.

Modify section -13.22- **Function `dispatch` [`async.dispatch`]** as follows.

-8- **Remarks:** This function shall not participate in overload resolution unless ~~`is_executor_v<Executor> is true`~~`is_convertible<Executor&, execution context&>::value is false`.

Modify section -13.23- **Function `post` [`async.post`]** as follows.

-8- **Remarks:** This function shall not participate in overload resolution unless ~~`is_executor_v<Executor> is true`~~`is_convertible<Executor&, execution context&>::value is false`.

Modify section -13.24- **Function `defer` [`async.defer`]** as follows.

-8- **Remarks:** This function shall not participate in overload resolution unless ~~`is_executor_v<Executor> is true`~~`is_convertible<Executor&, execution context&>::value is false`.

Modify section -13.25- **Class template `strand` [`async.strand`]** as follows.

-1- The class template `strand` is a wrapper around an object of type `Executor` satisfying the `execution::executor concept` (~~13.2.2~~`P0443R13`).

[...]

-2- `strand<Executor>` satisfies the ~~Executor (13.2.2) requirements~~`execution::executor concept (P0443R13)`.

Modify **Table 17 - AsyncReadStream requirements** [`buffer.stream.reqmts.asyncreadstream`] as follows:

operation	type	semantics, pre/post-conditions
<code>a.get_executor()</code>	A type satisfying the Executor requirements (13.2.2) <u><code>execution::executor concept (P0443R13)</code></u> .	Returns the associated I/O executor.

Modify **Table 19 - AsyncWriteStream requirements** [`buffer.stream.reqmts.asyncwritestream`] as follows:

operation	type	semantics, pre/post-conditions
<code>a.get_executor()</code>	A type satisfying the Executor requirements	Returns the associated

Specify the "Requirements on asynchronous operations" in terms of the new executors model

Modify section -13.2.7.10- *Outstanding work* [`async.reqmts.async.work`] as follows.

-1- Until the asynchronous operation has completed, the asynchronous operation shall maintain:

- ~~an object `work1` of type `executor_work_guard<Executor1>`, initialized as `work1(ex1)`, and where `work1.owns_work() == true`; and~~
- ~~an object `work2` of type `executor_work_guard<Executor2>`, initialized as `work2(ex2)`, and where `work2.owns_work() == true`;~~
- an executor object `work1`, initialized as `std::prefer(ex1, execution::outstanding_work.tracked)`; and
- an executor object `work2`, initialized as `std::prefer(ex2, execution::outstanding_work.tracked)`.

Modify section -13.2.7.12- *Execution of completion handler on completion of asynchronous operation* [`async.reqmts.async.completion`] as follows:

-3- If an asynchronous operation completes immediately (that is, within the thread of execution calling the initiating function, and before the initiating function returns), the completion handler shall be submitted for execution as if by performing ~~`ex2.post(std::move(f), alloc2)`~~. ~~Otherwise, the completion handler shall be submitted for execution as if by performing `ex2.dispatch(std::move(f), alloc2)`;~~

```
execution::execute(
std::prefer(
std::require(ex2, execution::blocking.never),
execution::allocator(alloc2)),
std::move(f));
```

Otherwise, the completion handler shall be submitted for execution as if by performing:

```
execution::execute(
std::prefer(work2, execution::blocking.possibly,
execution::allocator(alloc2)),
std::move(f));
```

Update `system_executor` and `system_context` to conform to the new executors model

Remove `system_executor` from, and add `system_context` to, -12.1- Header `<experimental/netfwd>` synopsis [`fwd.decl.synop`]:

```
class system_executor;
class system_context;
```

Update `system_executor` to be a type alias in -13.1- Header `<experimental/executor>` synopsis [`async.synop`]:

```
class system_executor;
class system_context;
using system_executor = system_context::executor_type;

bool operator==(const system_executor&, const system_executor&);
bool operator!=(const system_executor&, const system_executor&);
```

Remove section -13.18- *Class `system_executor`* [`async.system.exec`] in its entirety.

Modify section -13.19- *Class `system_context`* [`async.system.context`] as follows:

-1- Class `system_context` implements ~~the execution context associated with `system_executor` objects~~ an execution context that represents the ability to run a submitted function object on any thread.

```
namespace std {
namespace experimental {
namespace net {
inline namespace v1 {

    class system_context : public execution_context
    {
    public:
        // types:

        using executor_type = system_executorsee below;
```

[...]

-2- The class `system_context` satisfies the `ExecutionContext` (13.2.3) type requirements.

-?- `executor_type` is an executor type conforming to the specification for `system_context` executor types described below. Executor objects of type `executor_type` have the following properties established:

- `execution::blocking.possibly`
- `execution::relationship.fork`
- `execution::mapping.thread`
- `execution::allocator(std::allocator<void>())`.

-?- `system_context` executors having a different set of established properties may be represented by distinct, unspecified types. Function objects submitted via a `system_context` executor object are permitted to execute on any thread. To satisfy the requirements for the `execution::blocking.never` property, a `system_context` executor may create thread objects to run the submitted function objects. These thread objects are collectively referred to as system threads.

[...]

```
executor_type get_executor() noexcept;
```

-5- **Returns:** ~~`system_executor()`~~`executor_type()`.

After section -13.19- Class `system_context` [*async.system.context*] insert a new section as follows:

-13.-?- system context executor types

-1- All executor types accessible through `system_context::executor_type()`, `system_context::get_executor()`, and subsequent calls to the member function require, conform to the following specification.

```
namespace std {
namespace experimental {
namespace net {
inline namespace v1 {

    class system-context-executor
    {
    public:
    // construct / copy / destroy:

    system-context-executor(.) {}.

    // executor operations:
```

```

see below require(execution::blocking t::possibly t) const;
see below require(execution::blocking t::never t) const;
see below require(execution::blocking t::always t) const;
see below require(execution::relationship t::fork t) const;
see below require(execution::relationship t::continuation t) const;
see below require(execution::allocator t<void>) const;
template<class ProtoAllocator>
see below require(const execution::allocator t<ProtoAllocator>& a) const;

static constexpr execution::mapping t query(execution::mapping t) noexcept;
system context& query(execution::context t) const noexcept;
execution::blocking t query(execution::blocking t) const noexcept;
execution::relationship t query(execution::relationship t) const noexcept;
see below query(execution::allocator t<void>) const noexcept;
template<class ProtoAllocator>
see below query(const execution::allocator t<ProtoAllocator>&) const noexcept;

template<class Function>
void execute(Function&& f) const;
};

bool operator==(const system-context-executor& a, const system-context-executor& b) noexcept;
bool operator!=(const system-context-executor& a, const system-context-executor& b) noexcept;

} // inline namespace v1
} // namespace net
} // namespace experimental
} // namespace std

```

-2- system-context-executor is a type satisfying the execution::executor concept (P0443R13).

-13.?.1- system context executor operations

```

see below require(execution::blocking t::possibly t) const;
see below require(execution::blocking t::never t) const;
see below require(execution::blocking t::always t) const;
see below require(execution::relationship t::fork t) const;
see below require(execution::relationship t::continuation t) const;

```

-1- Returns: A system context executor object of an unspecified type conforming to these specifications, with the requested property established. When the requested property is part of a group that is defined as a mutually exclusive set, any other properties in the group are removed from the returned executor object. All other properties of the returned executor object are identical to those of *this.

```

see below require(execution::allocator t<void>) const;

```

-2- Returns: require(execution::allocator(std::allocator<void>(.))).

```

template<class ProtoAllocator>
see below require(const execution::allocator t<ProtoAllocator>& a) const;

```

-3- Returns: A system context executor object of an unspecified type conforming to these specifications, with the execution::allocator t<ProtoAllocator> property established such that allocation and deallocation associated with function submission will be performed using a copy of a.value(). All other properties of the returned executor object are identical to those of *this.

```

static constexpr execution::mapping t query(execution::mapping t) noexcept;

```

-4- Returns: true.

```

system context& query(execution::context t) const;

```


-5- **Returns:** A reference to the system context object.

execution::blocking_t query(execution::blocking_t) const noexcept;
execution::relationship_t query(execution::relationship_t) const noexcept;

-6- **Returns:** The established value of the property for the executor object *this.

see below query(execution::allocator t<void>) const noexcept;
template<class ProtoAllocator>
see below query(const execution::allocator t<ProtoAllocator>&) const noexcept;

-7- **Returns:** The allocator object associated with the executor, with type and value as previously established by the execution::allocator t<ProtoAllocator> property.

template<class Function>
void execute(Function&& f) const

-8- **Effects:** Submits the function f for execution according to the execution::executor concept and the properties established for *this. If f exits via an exception, calls std::terminate().

-13.?.2- system context executor comparisons

bool operator==(const system-context-executor& a, const system-context-executor& b) noexcept;

-1- **Returns:** true if a and b have identical properties, otherwise false.

bool operator!=(const system-context-executor& a, const system-context-executor& b) noexcept;

-2- **Returns:** !(a == b).

Remove the polymorphic wrapper executor and replace it with a type alias

Remove executor, and add a new type alias any_io_executor, in -12.1- Header <experimental/netfwd> synopsis [fwd.decl.synop]:

```
class executor;

namespace execution {
    template<class... SupportableProperties> class any_executor;
    template<class T> struct context as T;
    template<class T> struct prefer only;
} // namespace execution

using any_io_executor = execution::any_executor<
    execution::context as t<execution context>&,>,
    execution::blocking_t::never t,>,
    execution::prefer_only<execution::blocking_t::possibly t>,>,
    execution::prefer_only<execution::outstanding_work::untracked t>,>,
    execution::prefer_only<execution::outstanding_work::tracked t>,>,
    execution::prefer_only<execution::relationship_t::fork t>,>,
    execution::prefer_only<execution::relationship_t::continuation t>>;
```

Assuming the changes in P1322 have been applied, use any_io_executor as the default executor in -12.1- Header <experimental/netfwd> synopsis [fwd.decl.synop]:

```
template<class Clock> struct wait_traits;
template<class Clock, class WaitTraits = wait_traits<Clock>, class Executor = any_io_executor>
    class basic_waitable_timer;
using system_timer = basic_waitable_timer<chrono::system_clock>;
using steady_timer = basic_waitable_timer<chrono::steady_clock>;
using high_resolution_timer = basic_waitable_timer<chrono::high_resolution_clock>;
```



```

template<class Protocol, class Executor = any_io_executor>
    class basic_socket;
template<class Protocol, class Executor = any_io_executor>
    class basic_datagram_socket;
template<class Protocol, class Executor = any_io_executor>
    class basic_stream_socket;
template<class Protocol, class Executor = any_io_executor>
    class basic_socket_acceptor;
template<class Protocol, class Clock = chrono::steady_clock,
        class WaitTraits = wait_traits<Clock>, class Executor = any_io_executor>
    class basic_socket_streambuf;
template<class Protocol, class Clock = chrono::steady_clock,
        class WaitTraits = wait_traits<Clock>, class Executor = any_io_executor>
    class basic_socket_iostream;

namespace ip {

[...]
```

```

    template<class InternetProtocol, class Executor = any_io_executor>
        class basic_resolver;

```

Remove classes `bad_executor` and `executor`, and add a new type alias `any_io_executor`, in **-13.1- Header <experimental/executor> synopsis [async.synop]:**

```

class bad_executor;

class executor;

bool operator==(const executor& a, const executor& b) noexcept;
bool operator==(const executor& e, nullptr_t) noexcept;
bool operator==(nullptr_t, const executor& e) noexcept;
bool operator!=(const executor& a, const executor& b) noexcept;
bool operator!=(const executor& e, nullptr_t) noexcept;
bool operator!=(nullptr_t, const executor& e) noexcept;
using any_io_executor = execution::any_executor<
    execution::context as t<execution context>,>,,
    execution::blocking t::never t,,
    execution::prefer_only<execution::blocking t::possibly t>,,
    execution::prefer_only<execution::outstanding_work::untracked t>,,
    execution::prefer_only<execution::outstanding_work::tracked t>,,
    execution::prefer_only<execution::relationship t::fork t>,,
    execution::prefer_only<execution::relationship t::continuation t>>;

[...]
```

```

} // inline namespace v1
} // namespace net
} // namespace experimental

template<class Allocator>
struct uses_allocator<experimental::net::v1::executor, Allocator>
    : true_type {};

} // namespace std

```

Remove sections **-13.20- Class `bad_executor` [async.bad.exec]** and **-13.21- Class `executor` [async.executor]** in their entirety.

Assuming the changes in P1322 have been applied, use `any_io_executor` as the default executor in **-15.1- Header <experimental/timer> synopsis [timer.synop]:**

```
template<class Clock, class WaitTraits = wait_traits<Clock>, class Executor = any\_io\_executor>
    class basic_waitable_timer;
```

Assuming the changes in P1322 have been applied, use `any_io_executor` as the default executor in **-18.1- Header <experimental/socket> synopsis [socket.synop]:**

```
// Sockets:

class socket_base;

template<class Protocol, class Executor = any\_io\_executor>
    class basic_socket;

template<class Protocol, class Executor = any\_io\_executor>
    class basic_datagram_socket;

template<class Protocol, class Executor = any\_io\_executor>
    class basic_stream_socket;

template<class Protocol, class Executor = any\_io\_executor>
    class basic_socket_acceptor;

// [socket.iostreams], Socket streams:

template<class Protocol, class Clock = chrono::steady_clock,
    class WaitTraits = wait_traits<Clock>, class Executor = any\_io\_executor>
    class basic_socket_streambuf;

template<class Protocol, class Clock = chrono::steady_clock,
    class WaitTraits = wait_traits<Clock>, class Executor = any\_io\_executor>
    class basic_socket_istream;
```

Assuming the changes in P1322 have been applied, use `any_io_executor` as the default executor in **-21.1- Header <experimental/internet> synopsis [internet.synop]:**

```
template<class InternetProtocol, class Executor = any\_io\_executor>
    class basic_resolver;
```

Specify the free functions `dispatch`, `post`, and `defer` in terms of the new executors model

Modify section **-13.22- Function dispatch [async.dispatch]** as follows:

-1- [Note: The function dispatch satisfies the requirements for an asynchronous operation (13.2.7), except for the requirement that the operation uses `post`[the execution::blocking_never property](#) if it completes immediately. -- end note]

[...]

-3- **Effects:**

- Constructs an object completion of type `async_completion<CompletionToken, void()>`, initialized with token.
- ~~Performs `ex.dispatch(std::move(completion.completion_handler), alloc)`, where `ex` is the result of `get_associated_executor(completion.completion_handler)`, and `alloc` is the result of `get_associated_allocator(completion.completion_handler)`.~~
- [Performs:](#)

```
execution::execute\(
    std::prefer\(ex, execution::blocking.possibly,
        execution::allocator\(alloc\)\).,
    std::move\(completion.completion\_handler\)\).;
```

where ex is the result of get_associated_executor(completion.completion_handler), and alloc is the result of get_associated_allocator(completion.completion_handler).

[...]

-6- Effects:

- Constructs an object completion of type `async_completion<CompletionToken, void()>`, initialized with token.
- Constructs a function object `f` containing as members:
 - an executor object `w` for the completion handler's associated executor, initialized with `std::prefer(get_associated_executor(h), execution::outstanding_work)`,
 - a copy of the completion handler `h`, initialized with `std::move(completion.completion_handler)`,
 - ~~an executor_work_guard object `w` for the completion handler's associated executor, initialized with `make_work_guard(h)`;~~

and where the effect of `f()` is:

- ~~`w.get_executor().dispatch(std::move(h), alloc)`; where `alloc` is the result of `get_associated_allocator(h)`, followed by~~
- ~~`w.reset()`;~~

```
    execution::execute(  
        std::prefer(w, execution::blocking.possibly),  
        execution::allocator(alloc)),  
    std::move(h));
```

where `alloc` is the result of `get_associated_allocator(h)`.

- ~~Performs `ex.dispatch(std::move(f), alloc)`, where `alloc` is the result of `get_associated_allocator(completion.completion_handler)` prior to the construction of `f`;~~
- Performs:

```
    execution::execute(  
        std::prefer(ex, execution::blocking.possibly),  
        execution::allocator(alloc)),  
    std::move(f));
```

Modify section -13.23- *Function post [async.post]* as follows:

-3- Effects:

- Constructs an object completion of type `async_completion<CompletionToken, void()>`, initialized with token.
- ~~Performs `ex.post(std::move(completion.completion_handler), alloc)`, where `ex` is the result of `get_associated_executor(completion.completion_handler)`, and `alloc` is the result of `get_associated_allocator(completion.completion_handler)`;~~
- Performs:

```
    execution::execute(  
        std::prefer(  
            std::require(ex, execution::blocking.never),  
            execution::relationship.fork, execution::allocator(alloc)),  
        std::move(completion.completion_handler));
```

where ex is the result of get_associated_executor(completion.completion_handler), and alloc is the result of get_associated_allocator(completion.completion_handler).

[...]

-6- Effects:

- Constructs an object completion of type `async_completion<CompletionToken, void()>`, initialized with token.
- Constructs a function object `f` containing as members:
 - an executor object `w` for the completion handler's associated executor, initialized with `std::prefer(get_associated_executor(h), execution::outstanding_work)`,
 - a copy of the completion handler `h`, initialized with `std::move(completion.completion_handler)`,
 - ~~an executor_work_guard object `w` for the completion handler's associated executor, initialized with `make_work_guard(h)`;~~

and where the effect of `f()` is:

- ~~`w.get_executor().dispatch(std::move(h), alloc)`, where `alloc` is the result of `get_associated_allocator(h)`, followed by~~
- ~~`w.reset()`;~~

```
    execution::execute(
        std::prefer(w, execution::blocking.possibly),
        execution::allocator(alloc)),
    std::move(h));
```

where `alloc` is the result of `get_associated_allocator(h)`.

- ~~Performs `ex.dispatch(std::move(f), alloc)`, where `alloc` is the result of `get_associated_allocator(completion.completion_handler)` prior to the construction of `f`;~~
- Performs:

```
    execution::execute(
        std::prefer(
            std::require(ex, execution::blocking.never),
            execution::relationship.fork, execution::allocator(alloc)),
        std::move(f));
```

Modify section -13.24- **Function `defer` [`async.defer`]** as follows:

-1- [Note: The function `defer` satisfies the requirements for an asynchronous operation (13.2.7), ~~except for the requirement that the operation uses `post` if it completes immediately.~~ --end note]

[...]

-3- Effects:

- Constructs an object completion of type `async_completion<CompletionToken, void()>`, initialized with token.
- ~~Performs `ex.defer(std::move(completion.completion_handler), alloc)`, where `ex` is the result of `get_associated_executor(completion.completion_handler)`, and `alloc` is the result of `get_associated_allocator(completion.completion_handler)`;~~
- Performs:

```
    execution::execute(
        std::prefer(
            std::require(ex, execution::blocking.never),
            execution::relationship.continuation, execution::allocator(alloc)),
        std::move(completion.completion_handler));
```

where `ex` is the result of `get_associated_executor(completion.completion_handler)`, and `alloc` is the result of `get_associated_allocator(completion.completion_handler)`.

[...]

-6- Effects:

- Constructs an object completion of type `async_completion<CompletionToken, void()>`, initialized with token.

- Constructs a function object `f` containing as members:
 - an executor object `w` for the completion handler's associated executor, initialized with `std::prefer(get_associated_executor(h), execution::outstanding_work)`,
 - a copy of the completion handler `h`, initialized with `std::move(completion.completion_handler)`,
 - ~~an executor_work_guard object `w` for the completion handler's associated executor, initialized with `make_work_guard(h)`;~~

and where the effect of `f()` is:

- ~~`w.get_executor().dispatch(std::move(h), alloc)`, where `alloc` is the result of `get_associated_allocator(h)`, followed by~~
- ~~`w.reset()`;~~

```
    execution::execute(
    std::prefer(w, execution::blocking.possibly,
    execution::allocator(alloc)).,
    std::move(h)).;
```

where `alloc` is the result of `get_associated_allocator(h)`.

- ~~Performs `ex.dispatch(std::move(f), alloc)`, where `alloc` is the result of `get_associated_allocator(completion.completion_handler)` prior to the construction of `f`;~~
- Performs:

```
    execution::execute(
    std::prefer(
        std::require(ex, execution::blocking.never),
        execution::relationship.continuation, execution::allocator(alloc)).,
    std::move(f)).;
```

Update the strand adapter to conform to the new executors model

Modify section -13.25- *Class template strand [async.strand]* as follows:

```
namespace std {
namespace experimental {
namespace net {
inline namespace v1 {
```

```
    template<class Executor>
    class strand
    {
    public:
```

[...]

```
    execution_context& context() const noexcept;
```

```
    void on_work_started() const noexcept;
```

```
    void on_work_finished() const noexcept;
```

```
    template<class Func, class ProtoAllocator>
```

```
    void dispatch(Func&& f, const ProtoAllocator& a) const;
```

```
    template<class Func, class ProtoAllocator>
```

```
    void post(Func&& f, const ProtoAllocator& a) const;
```

```
    template<class Func, class ProtoAllocator>
```

```
    void defer(Func&& f, const ProtoAllocator& a) const;
```

```
    template<class Property>
```

```
    see below query(const Property& p) const;
```

```
    template<class Property>
```

```
    see below require(const Property& p) const;
```

```
    template<class Property>
```

see below prefer(const Property& p) const;

template<class Function>
void execute(Function&& f) const;

[...]

-3- A strand provides guarantees of ordering and non-concurrency. Given:

- strand objects s1 and s2 such that s1 == s2
- a function object f1 ~~added~~submitted to the strand s1 ~~using post or defer, or using dispatch~~when the execution::blocking.never property is established in s1, or when s1.running_in_this_thread() is false
- a function object f2 ~~added~~submitted to the strand s2 ~~using post or defer, or using dispatch~~when the execution::blocking.never property is established in s2, or when s2.running_in_this_thread() is false

Modify section -13.25.4- **strand operations [async.strand.ops]** as follows:

bool running_in_this_thread() const noexcept;

-2- **Returns:** true if the current thread of execution is running a function that was submitted to the strand, or to any other strand object ~~s such that s == *this, using dispatch, post or defer~~that shares the same ordered, non-concurrent state, using execute; otherwise false. [Note: That is, the current thread of execution's call chain includes a function that was submitted to the strand. --end note]

~~void on_work_started() const noexcept;~~

~~-4- **Effects:** Calls inner_ex_.on_work_started();~~

~~void on_work_finished() const noexcept;~~

~~-5- **Effects:** Calls inner_ex_.on_work_finished();~~

~~template<class Func, class ProtoAllocator>~~
~~void dispatch(Func&& f, const ProtoAllocator& a) const;~~

~~-6- **Effects:** If running_in_this_thread() is true, calls DECAY_COPY(forward<Func>(f))(). (C++ 2014 [thread.decaycopy]). [Note: If f exits via an exception, the exception propagates to the caller of dispatch(). --end note] Otherwise, requests invocation of f, as if by forwarding the function object f and allocator a to the executor inner_ex_, such that the guarantees of ordering and non-concurrency are met.~~

~~template<class Func, class ProtoAllocator>~~
~~void post(Func&& f, const ProtoAllocator& a) const;~~

~~-7- **Effects:** Requests invocation of f, as if by forwarding the function object f and allocator a to the executor inner_ex_, such that the guarantees of ordering and non-concurrency are met.~~

~~template<class Func, class ProtoAllocator> void defer(Func&& f, const ProtoAllocator& a) const;~~

~~-8- **Effects:** Requests invocation of f, as if by forwarding the function object f and allocator a to the executor inner_ex_, such that the guarantees of ordering and non-concurrency are met.~~

template<class Property>
see below query(const Property& p) const;

-?- **Returns:** std::query(inner_ex_, p).

-?- **Remarks:** Shall not participate in overload resolution unless `can_query v<const Executor&, const Property&>` is true.

`template<class Property>`
`see below require(const Property& p) const;`

-?- **Returns:** A strand `s` of type `strand<decay_t<decltype(std::require(inner_ex_,_p))>>`, where `s.inner_ex` is initialized with `std::require(inner_ex_,_p)`, and sharing the same ordered, non-concurrent state as `*this`.

-?- **Remarks:** Shall not participate in overload resolution unless `can_require v<const Executor&, const Property&>` is true.

`template<class Property>`
`see below prefer(const Property& p) const;`

-?- **Returns:** A strand `s` of type `strand<decay_t<decltype(std::prefer(inner_ex_,_p))>>`, where `s.inner_ex` is initialized with `std::prefer(inner_ex_,_p)`, and sharing the same ordered, non-concurrent state as `*this`.

-?- **Remarks:** Shall not participate in overload resolution unless `can_prefer v<const Executor&, const Property&>` is true.

`template<class Function>`
`void execute(Function&& f) const;`

-?- **Effects:** Submits `f` to the executor `inner_ex_`, such that the guarantees of ordering and non-concurrency are met.

Update the `use_future` completion token to conform to the new executors model

Modify section -13.26.2- *use_future_t* members [*async.use.future.members*] as follows:

-8- For any executor type `E`, the associated object for the associator `associated_executor<H, E>` is an executor where, for function objects executed using the executor's ~~`dispatch()`, `post()` or `defer()` functions~~`execute()` function, any exception thrown is caught by a function object and stored in the associated shared state.

Modify section -13.26.3- *Partial class template specialization `async_result` for `use_future_t`* [*async.use.future.result*] as follows:

-3- The implementation specializes `associated_executor` for `F`. For function objects executed using the associated executor's ~~`dispatch()`, `post()` or `defer()` functions~~`execute()` function, any exception thrown is caught by the executor and stored in the associated shared state.

-4- For any executor type `E`, the associated object for the associator `associated_executor<F, E>` is an executor where, for function objects executed using the executor's ~~`dispatch()`, `post()` or `defer()` functions~~`execute()` function, any exception thrown by a function object is caught by the executor and stored in the associated shared state.

Update `io_context` to conform to the new executors model

Modify section -14.2- *Class `io_context`* [*io_context.io_context*] as follows:

```
namespace std {
namespace experimental {
namespace net {
inline namespace v1 {
```



```

class io_context : public execution_context
{
public:
    // types:

    class executor_type;
    using executor_type = see below;

```

[...]

-1- The class `io_context` satisfies the `ExecutionContext` type requirements (13.2.3).

-?- `executor_type` is an executor type conforming to the specification for `io_context` executor types described below.

[...]

-4- For an object of type `io_context`, *outstanding work* is defined as the sum of:

- ~~• the total number of calls to the `on_work_started` function, less the total number of calls to the `on_work_finished` function, to any executor of the `io_context`;~~
- the number of existing executor objects associated with the `io_context` for which the `execution::outstanding_work.tracked` property is established;
- the number of function objects that have been added to the `io_context` via any executor of the `io_context`, but not yet executed; and
- the number of function objects that are currently being executed by the `io_context`.

[...]

```

executor_type get_executor() noexcept;

```

-3- **Returns:** An executor that may be used for submitting function objects to the `io_context`. The returned executor has the following properties already established:

- `execution::blocking.possibly`
- `execution::relationship.fork`
- `execution::outstanding_work.untracked`
- `execution::mapping.thread`
- `execution::allocator(std::allocator<void>())`

-?- `io_context` executors having a different set of established properties may be represented by distinct, unspecified types.

[...]

-13- **Remarks:** This function may invoke additional function objects through nested calls to the `io_context` executor's ~~`dispatch`~~`execute` member function. These do not count towards the return value.

[...]

-19- **Remarks:** This function may invoke additional function objects through nested calls to the `io_context` executor's ~~`dispatch`~~`execute` member function. These do not count towards the return value.

[...]

-25- **Remarks:** This function may invoke additional function objects through nested calls to the `io_context` executor's ~~`dispatch`~~`execute` member function. These do not count towards the return value.

Remove section -14.3- Class `io_context::executor_type` [`io_context.exec`] in its entirety.

After section -14.2- Class `io_context` [`io_context.io_context`] insert a new section as follows:

-14.?- io context executor types

All executor types accessible through `io_context::get_executor()`, and subsequent calls to the member function `require`, conform to the following specification.

```
namespace std {
namespace experimental {
namespace net {
inline namespace v1 {

    class io-context-executor
    {
    public:
        // construct / copy / destroy:

        io-context-executor(const io-context-executor& other) noexcept;
        io-context-executor(io-context-executor&& other) noexcept;

        io-context-executor& operator=(const io-context-executor& other) noexcept;
        io-context-executor& operator=(io-context-executor&& other) noexcept;

        // executor operations:

        see below require(execution::blocking t::possibly t) const;
        see below require(execution::blocking t::never t) const;
        see below require(execution::relationship t::fork t) const;
        see below require(execution::relationship t::continuation t) const;
        see below require(execution::outstanding work t::untracked t) const;
        see below require(execution::outstanding work t::tracked t) const;
        see below require(execution::allocator t<void>) const;
        template<class ProtoAllocator>
            see below require(const execution::allocator t<ProtoAllocator>& a) const;

        static constexpr execution::mapping t query(execution::mapping t) noexcept;
        io context& query(execution::context t) const noexcept;
        execution::blocking t query(execution::blocking t) const noexcept;
        execution::relationship t query(execution::relationship t) const noexcept;
        execution::outstanding work t query(execution::outstanding work t) const noexcept;
        see below query(execution::allocator t<void>) const noexcept;
        template<class ProtoAllocator>
            see below query(const execution::allocator t<ProtoAllocator>&) const noexcept;

        bool running_in_this_thread() const noexcept;

        template<class Function>
            void execute(Function&& f) const;
    };

    bool operator==(const io-context-executor& a, const io-context-executor& b) noexcept;
    bool operator!=(const io-context-executor& a, const io-context-executor& b) noexcept;

} // inline namespace v1
} // namespace net
} // namespace experimental
} // namespace std
```

-1- `io-context-executor` is a type satisfying the `execution::executor` concept (P0443R13). Objects of type `io-context-executor` are associated with an `io_context`, and function objects submitted using the `execute` member function will be executed by the `io_context` from within a `run` function.

-14.?.1- io context executor constructors

io-context-executor(const io-context-executor& other) noexcept;

-1- **Postconditions:** *this == other.

io-context-executor(io-context-executor&& other) noexcept;

-2- **Postconditions:** *this is equal to the prior value of other.

-14.?.2- io context executor assignment

io-context-executor& operator=(const io-context-executor& other) noexcept;

-1- **Postconditions:** *this == other.

-2- **Returns:** *this.

io-context-executor& operator=(io-context-executor&& other) noexcept;

-3- **Postconditions:** *this is equal to the prior value of other.

-4- **Returns:** *this.

-14.?.3- io context executor operations

see below require(execution::blocking t::possibly t) const;
see below require(execution::blocking t::never t) const;
see below require(execution::relationship t::fork t) const;
see below require(execution::relationship t::continuation t) const;
see below require(execution::outstanding work t::untracked t) const;
see below require(execution::outstanding work t::tracked t) const;

-1- **Returns:** An executor object of an unspecified type conforming to these specifications, associated with the same io context as *this, and having the requested property established. When the requested property is part of a group that is defined as a mutually exclusive set, any other properties in the group are removed from the returned executor object. All other properties of the returned executor object are identical to those of *this.

see below require(execution::allocator t<void>) const;

-2- **Returns:** require(execution::allocator(std::allocator<void>(.))).

template<class ProtoAllocator>

see below require(const execution::allocator t<ProtoAllocator>& a) const;

-3- **Returns:** An executor object of an unspecified type conforming to these specifications, associated with the same io context as *this, with the execution::allocator t<ProtoAllocator> property established such that allocation and deallocation associated with function submission will be performed using a copy of a.value(). All other properties of the returned executor object are identical to those of *this.

static constexpr execution::mapping_t query(execution::execution::mapping_t) noexcept;

-4- **Returns:** true.

io context& query(execution::context t) const noexcept;

-5- **Returns:** A reference to the associated io context object.

execution::blocking_t query(execution::blocking_t) const noexcept;
execution::relationship_t query(execution::relationship_t) const noexcept;
execution::outstanding work_t query(execution::outstanding work_t) const noexcept;

-6- **Returns:** The established value of the property for the executor object **this*.

see below query(execution::allocator t<void>) const noexcept;
template<class ProtoAllocator>
see below query(const execution::allocator t<ProtoAllocator>&) const noexcept;

-7- **Returns:** The allocator object associated with the executor, with type and value as previously established by the execution::allocator t<ProtoAllocator> property.

bool running_in_this_thread() const noexcept;

-8- **Returns:** true if the current thread of execution is calling a run function of the associated io context object. [Note: That is, the current thread of execution's call chain includes a run function. --end note]

template<class Function>
void execute(Function&& f) const

-9- **Effects:** Submits the function f for execution on the io context according to the execution::executor concept and the properties established for **this*. If f exits via an exception, the exception does not propagate to the caller of execute(), but is instead subsequently propagated to a caller of a run function for the io context object.

-14.?4- io context executor comparisons

bool operator==(const io-context-executor& a, const io-context-executor& b) noexcept;

-1- **Returns:** true if addressof(a.query(execution::context_t)) == addressof(b.query(execution::context_t)), and a and b have identical properties, otherwise false.

bool operator!=(const io-context-executor& a, const io-context-executor& b) noexcept;

-2- **Returns:** !(a == b).

Proposed Extension to P0443R13 A Unified Executors Proposal for C++

The asynchronous operation requirements specified above stipulate that querying the execution::context_t property for Networking TS executors returns a type of either execution_context&, or of E& where E is a type that is unambiguously derived from execution_context.

However, the execution::context_t property as currently specified in P0443 uses std::any as its polymorphic_query_result_type. This prevents the polymorphic wrapper execution::any_io_executor from satisfying the above requirements.

For this reason, this paper proposes the following extension to P0443R13.

Add the following context_as_t<> property adapter. This adapter allows us to expose the execution::context_t property through the polymorphic wrapper using a polymorphic query type of our choosing.

-?- Struct context_as_t

-?- The context_as_t struct is a property adapter for the context_t property, to specify a polymorphic_query_result_type.

-?- [Example: context_as_t may be used with the polymorphic wrapper executor to expose the query-only property context_t using a suitable polymorphic type:

```

static thread pool pool;

execution::any_executor<
    execution::context as t<static thread pool&>
> my_executor(pool.executor());

// ...

static thread pool& ctx =
    std::query(my_executor, execution::context);

--end example]

template<class T>
struct context as t
{
    template <typename U>
    static constexpr bool is_applicable_property_v
        = executor<U> || sender<U> || scheduler<U>;

    static constexpr bool is_requirable = false;
    static constexpr bool is_preferable = false;

    using polymorphic_query_result_type = T;

    template<class Executor>
    static constexpr T static_query_v
        = context t::static_query_v<Executor>;

    constexpr context as t() {}.
    constexpr context as t(execution::context t) {}.

    template<class Executor, class Property>
    friend constexpr T query(const Executor& ex, const Property& p).
    noexcept(noexcept(std::query(ex, execution::context))).
    -> decltype(std::query(ex, execution::context));
};

template<class T>
constexpr context as t context as;

```

-?- The expression `context as t<T>::static_query_v<E>` is well-formed for some executor type `E` if and only if the expression `context t::static_query_v<E>` is well-formed and can be used to initialize a constant of type `T`.

```

template<class Executor, class Property>
friend constexpr T query(const Executor& ex, const Property& p).
    noexcept(noexcept(std::query(ex, execution::context))).

```

-?- **Returns:** `std::query(ex, p.property)`.

-?- **Remarks:** Shall not participate in overload resolution unless `std::is_same_v<Property, context as t>` is true, and the expression `std::query(ex, execution::context)` is well-formed.

Revision History

The following changes were made in revision 2 of this paper:

- Updated to match the unified executor facilities as described in P0443R13.
- Updated to use property facilities from namespace `std` rather than `std::execution`.
- No longer propose the addition of the `context()` member function to I/O objects.
- Renamed the polymorphic wrapper type alias to `any_io_executor`.
- Added `any_io_executor` to the forward declaration header `<netfwd>`.

- Added changes to use `any_io_executor` as the default, assuming P1322 changes have been applied.
- Renamed `context_as` to `context_as_t`, and added template variable `context_as`.
- Updated `system_context` executor specification based on implementation experience.
- Updated `io_context` executor specification based on implementation experience.

The following changes were made in revision 1 of this paper:

- Updated to match the unified executor facilities as described in P0443R7.
- Added a section on deployment experience.
- Specify that `system_context` executors use distinct types when different properties are established.
- Added missing property queries on `system_context` executors.
- Changed `strand` to conditionally forward static `query()` calls to the adapted executor.
- Specify that `io_context` executors use distinct types when different properties are established.
- Added missing property queries on `io_context` executors.
- Added modifications to P0443R7 executors, including the `execution::context_as<>` property adapter.